

NAME : SAMIKSHA SHIVAJI BHOSALE

PRN : 202501050011

BATCH : CH1

PRACTICAL 4

PRACTICAL 4.1.1 PANDAS-SERIES CREATION AND MANIPULATION

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

CODE :

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

```
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

The screenshot displays the CDETANTRA IDE interface. The Explorer panel on the left shows a file named 'seriesMa...'. The main editor area contains a Python script with the following code:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
```

The Output panel on the right shows the execution results. It includes a table of sample test cases and the final output of the script. The test cases are:

Test case 1
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers: 5.0
Odd: 5.0
Even: 6.0
dtype: float64

The final output of the script is:

```
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers: 5.0
Odd: 5.0
Even: 6.0
dtype: float64
```

PRACTICAL 4.1.2 DICTIONARY TO DATAFRAME

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.

- Display the DataFrame after deleting the column.

CODE :

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
new_name = input("New name: ")
```

```
new_age = int(input("New age: "))
```

```
new_row = {'Name': new_name, 'Age': new_age }
```

```
df = df.append(new_row,ignore_index = True)
```

```
# Display the DataFrame after adding a new row
```

```
print("After adding a row:\n",df)
```

```
# Modifying a row
```

```
modify_index = int(input("Index of row to modify: "))  
new_age = int(input("New age: "))  
df.at[modify_index,'Age'] = new_age
```

```
# Display the DataFrame after modifying a row  
print("After modifying a row:")  
print(df)
```

```
# Deleting a row  
del_index = int(input("Index of row to delete: "))  
df = df.drop(del_index).reset_index(drop = True)  
# Display the DataFrame after deleting a row  
print("After deleting a row:")  
print(df)
```

```
# Adding a new column  
genders = input("Enter genders separated by space: ").split()  
df["Gender"] = genders  
# Display the DataFrame after adding a new column  
print("After adding a new column:")  
print(df)
```

```
# Modifying a column  
df['Name'] = df['Name'].str.upper()  
# Display the DataFrame after modifying a column  
print("After modifying a column:")
```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = 'Age').reset_index(drop = True)
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

The screenshot shows the CODETANTRA IDE interface. On the left, a sidebar displays the file explorer and a list of test cases. The main editor area shows a Python script that creates a DataFrame from a dictionary and then deletes the 'Age' column. The script is as follows:

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
```

Below the script, the test results are shown. The test case 'Test case 1' is highlighted, and the output shows the DataFrame after deleting the 'Age' column. The output is as follows:

```
Original DataFrame:
  Name  Age
0  Alice  25
1  Bob   30
2  Charlie 35

New name: Susan
New age: 29

After adding a row:
  Name  Age
0  Alice  25
1  Bob   30
```

The test results indicate that 1 out of 1 shown test case(s) passed and 1 out of 1 hidden test case(s) passed. The average time for the test case is 0.067 s, and the maximum time is 0.078 s.

PRACTICAL 4.1.3 STUDENT INFORMATION

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

CODE :

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# write your code here..
```

```
print("First five rows:")
```

```
print(data.head())
```

```
av = round(data['Age'].mean(),2)
```

```
print("Average age:",av)
```

```
filtered_data = data[data['Grade'].isin(['A','B'])]
```

```
print("Students with a grade up to B")
```

```
print(filtered_data)
```

The screenshot displays the CODETANTRA IDE interface. On the left, a sidebar shows the project structure with a file named 'studentdata.txt'. The main editor area contains a Python script that reads a CSV file into a DataFrame, prints the first five rows, calculates the average age, and filters students by grade. The script is as follows:

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

# write your code here..
print("First five rows:")
print(data.head())

av = round(data['Age'].mean(),2)
print("Average age:",av)

filtered_data = data[data['Grade'].isin(['A','B'])]
print("Students with a grade up to B")
print(filtered_data)
```

Below the code editor, the test case runner shows the results of the execution. It indicates that 1 out of 1 shown test case(s) passed and 1 out of 1 hidden test case(s) passed. The test case details include the input file 'studentdata.txt' and the expected output, which is the first five rows of the DataFrame:

Name	Age	Grade
John	25	A
Alin	22	B
Emma	24	A
Mike	23	C
Sony	21	B

The terminal at the bottom shows the output of the script, which matches the expected output of the test case.

PRACTICAL 4.2.1 MONTH WITH THE HIGHEST TOTAL SALES

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
df['Total Sales'] = df['Quantity'] * df['Price']
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df['Month'] = df['Date'].dt.to_period('M')
```

```
monthly_sales = df.groupby('Month')['Total Sales'].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")
```

```
print(f"Total sales: ${highest_sales:.2f}")
```

CODETANTRA Home Learn Anywhere 202501050011@mitaoe.ac.in Support Logout

4.2.1. Month with the Highest Total Sales

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df['Total Sales'] = df['Quantity'] * df['Price']
9 df['Date'] = pd.to_datetime(df['Date'])
10 df['Month'] = df['Date'].dt.to_period('M')
11 monthly_sales = df.groupby('Month')['Total Sales'].sum()
```

Average time: 0.022 s (22.00 ms) Maximum time: 0.045 s (45.00 ms)

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (45 ms) Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

Terminal Test Cases

< Prev Reset Submit Next >

PRACTICAL 4.2.2 BEST SELLING PRODUCT

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
ps = df.groupby("Product")["Quantity"].sum()
```

```
# Find the product with the highest total quantity sold
```

```
best_product = ps.idxmax()
```

```
highest_quantity = ps.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```

The screenshot displays the CODETANTRA IDE interface. On the left, a 'Sample Data' table is shown with columns: Date, Product, Quantity, Price, City. The data includes sales for Product A and Product B across different dates and cities. Below the table, a 'Note' states: 'The data cannot be displayed in the file. You can refer to the sample data provided for insights.' Underneath, 'Sample Test Cases' are listed, including 'Test case 1' with 'Expected output' and 'Actual output' fields.

The main code editor shows the following Python code:

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 ps = df.groupby("Product")["Quantity"].sum()
9
10 # Find the product with the highest total quantity sold
11 best_product = ps.idxmax()
```

Below the code editor, test case results are displayed. 'Test case 1' is shown with a '35 ms' execution time. The 'Expected output' and 'Actual output' fields both show 'sales_data.csv'. The 'Best selling product' is 'Product A' and the 'Total quantity sold' is '23'.

The bottom of the interface features a 'Terminal' and 'Test cases' tab, along with navigation buttons: '< Prev', 'Reset', 'Submit', and 'Next >'.

PRACTICAL 4.2.3 CITY THAT SOLD THE MOST PRODUCTS

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data

df = pd.read_csv(file_name)

# write the code..

cs = df.groupby("City")["Quantity"].sum()

best_city = cs.idxmax()

# Display the result

print(f"City sold the most products: {best_city}")
```

The screenshot displays a coding environment with the following components:

- Problem Statement:** "4.2.3. City that Sold the Most Products".
- Sample Data:** A table with columns: Date, Product, Quantity, Price, City. The data shows sales for various products across different dates and cities.
- Note:** "The data cannot be displayed in the file. You can refer to the sample data provided for insights."
- Sample Test Cases:** A section showing a test case where the input is "sales_data.csv" and the expected output is "City sold the most products: Los Angeles".
- Code Editor:** A Python script that reads a CSV file, groups the data by city, and finds the city with the maximum quantity sold.
- Test Results:** A summary showing that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed.
- Terminal:** A section for running the code and viewing the output.

PRACTICAL 4.2.4 MOST FREQUENTLY SOLD PRODUCT PAIRS

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)

- Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd

from itertools import combinations

from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
daily_transactions = df.groupby('Date')['Product'].apply(list)
pair_counts = Counter()

for products in daily_transactions:
    products = sorted(products)
    pairs = list(combinations(products, 2))
    pair_counts.update(pairs)

# Output the most frequent product pairs
if pair_counts:
```

```
max_count = max(pair_counts.values())
```

```
for pair, count in pair_counts.items():
```

```
    if count == max_count:
```

```
        print(f"{pair[0]} and {pair[1]}: {count} times")
```

```
    else:
```

```
        print("No products pairs found.")
```

The screenshot shows the CodeTANTRA IDE interface. On the left, a sidebar displays the file explorer with a file named 'frequentl...'. The main editor area shows a Python script for finding frequently sold product pairs. The script uses pandas, itertools, and collections. It prompts the user for a file name, reads a CSV file, and prints the results. The interface includes a file explorer, a code editor, a terminal, and a test cases panel. The test cases panel shows that the script has passed 2 out of 2 hidden test cases.

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
```

Test case 1 (37 ms) [Debug] [Test cases]

Expected output	Actual output
sales_data.csv	sales_data.csv
Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times

PRACTICAL 4.2.5 TITANIC DATASET ANALYSIS AND DATA CLEANING

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# 1. Display the first 5 rows of the dataset
```

```
print(data.head())
```

```
# 2. Display the last 5 rows of the dataset
```

```
print(data.tail())
```

```
# 3. Get the shape of the dataset
```

```
print(data.shape)
```

```
# 4. Get a summary of the dataset (info)
```

```
print(data.info())
```

```
# 5. Get basic statistics of the dataset
```

```
print(data.describe())
```

```
# 6. Check for missing values
```

```
print(data.isnull().sum())
```

7. Fill missing values in the 'Age' column with the median age

```
data['Age'].fillna(data['Age'].median(), inplace = True)
```

8. Fill missing values in the 'Embarked' column with the mode

```
data['Embarked'].fillna(data['Embarked'].mode()[0],inplace=True)
```

9. Drop the 'Cabin' column due to many missing values

```
data.drop(columns='Cabin',inplace = True)
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

```
data["FamilySize"] = data['SibSp']+data['Parch']
```

The screenshot displays the CODETANTRA IDE interface. The top navigation bar includes the logo, home link, and user information. The main workspace is divided into three panels: a left sidebar with a file explorer and test cases, a central code editor, and a right sidebar with a terminal and test results.

The code editor shows a Python script for Titanic dataset analysis and data cleaning. The script includes imports for pandas and numpy, loading the dataset, and displaying the first and last 5 rows.

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
```

The test cases section shows a single test case passed. The output section displays the expected and actual data for the first 5 rows.

PassengerId	Survived	Pclass	Fare	Cabin	Embarked
0	0	3	7.2500	NaN	S
1	1	1	71.2833	C85	C
2	1	3	7.9250	NaN	S
3	1	1	53.1000	C123	S
4	0	3	8.0500	NaN	S

PRACTICAL 4.2.6 TITANIC DATASET ANALYSIS AND DATA CLEANING-2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
data['Sex'] = data['Sex'].map({'male':0, 'female':1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
data = pd.get_dummies(data, columns=["Embarked"])
```

```
# 4. Get the mean age of passengers
```

```
mean_age = data['Age'].mean()
```

```
print(mean_age)
```

5. Get the median fare of passengers

```
median_fare = data['Fare'].median()
```

```
print(median_fare)
```

6. Get the number of passengers by class

```
print(data['Pclass'].value_counts())
```

7. Get the number of passengers by gender

```
print(data['Sex'].value_counts())
```

8. Get the number of passengers by survival status

```
print(data['Survived'].value_counts())
```

9. Calculate the survival rate

```
survival_rate = data['Survived'].mean()
```

```
print(format(survival_rate))
```

10. Calculate the survival rate by gender

```
survival_by_gender = data.groupby('Sex')['Survived'].mean()
```

```
print(survival_by_gender)
```

The screenshot displays the CODETANTRA web interface for a Python programming task. The interface is divided into several sections:

- Header:** CODETANTRA logo, navigation links (Home, Learn Anywhere), user information (202501050011@mitaoe.ac.in), and a Logout button.
- Task Title:** 4.2.6. Titanic Dataset Analysis and Data Cleaning - 2.
- Code Editor:** Contains the following Python code:


```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
```
- Test Case Runner:** Shows the execution results for Test case 1. It indicates that the code passed the test case with an average time of 0.061 s and a maximum time of 61.00 ms. The output is displayed in a table:

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
- Debugger:** A section for debugging the code, currently showing the Test cases tab.

PRACTICAL 4.2.7 TITANIC DATASET ANALYSIS AND DATA CLEANING-3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']  
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)  
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

1. Calculate the survival rate by class

```
print(data.groupby('Pclass')['Survived'].mean())
```

2. Calculate the survival rate by embarkation location (Embarked_S)

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

3. Calculate the survival rate by family size

```
print(data.groupby('FamilySize')['Survived'].mean())
```

4. Calculate the survival rate by being alone

```
print(data.groupby('IsAlone')['Survived'].mean())
```

5. Get the average fare by passenger class

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by passenger class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

Filter for Survived=1, then count occurrences of Pclass

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

Filter for Survived=0, then count occurrences of Pclass

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

The screenshot displays a Jupyter Notebook environment with a dark theme. The top navigation bar includes the 'CDE TANTRA' logo, a 'Home' link, a 'Learn Anywhere' dropdown, a user email '202501050011@mitaoe.ac.in', a 'Support' link, and a 'Logout' button. The notebook title is '4.2.7. Titanic Dataset Analysis and Data Cleaning - 3'. The left sidebar shows 'Sample Test Cases' for 'Test case 1', listing input parameters like 'Pclass' and 'Survived' with their respective values and data types. The main area contains a code editor with Python code for loading the Titanic dataset, calculating family size, and determining survival rates by class. The code is as follows:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
```

Below the code editor, the execution results are shown. The 'Average time' is 0.071 s and the 'Maximum time' is 71.00 ms. A green checkmark indicates '1 out of 1 shown test case(s) passed'. The 'Test case 1' results are displayed in a table with 'Expected output' and 'Actual output' columns, showing the survival rate for each Pclass (1, 2, 3).

Pclass	Survived
1	0.629630
2	0.472826
3	0.242363

PRACTICAL 4.2.8 TITANIC DATASET ANALYSIS AND DATA CLEANING-4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
print(data[data['Survived']==1]['Sex'].value_counts())
```

```
# 2. Get the number of non-survivors by gender
```

```
print(data[data['Survived']==0]['Sex'].value_counts())
```

```
# 3. Get the number of survivors by embarked location
```

```
print(data[data['Survived']==1]['Embarked_S'].value_counts())
```

```
# 4. Get the number of non-survivors by embarked location
```

```
print(data[data['Survived']==0]['Embarked_S'].value_counts())
```

```
# 5. Calculate the percentage of children (Age < 18) who survived
```

```
print(data[data['Age']<18]['Survived'].mean())
```

6. Calculate the percentage of adults (Age = 18) who survived

```
print(data[data['Age']>=18]['Survived'].mean())
```

7. Get the median age of survivors

```
print(data[data['Survived']==1]['Age'].median())
```

8. Get the median age of non-survivors

```
print(data[data['Survived']==0]['Age'].median())
```

9. Get the median fare of survivors

```
print(data[data['Survived']==1]['Fare'].median())
```

10. Get the median fare of non-survivors

```
print(data[data['Survived']==0]['Fare'].median())
```

The screenshot displays the CODETANTRA IDE interface. The top bar shows the logo, navigation links, and user information. The main workspace is divided into three panels: a file explorer on the left, a code editor in the center, and a test runner on the right.

Code Editor: The code defines a function `titanicData` that loads the Titanic dataset, creates dummy variables for the 'Embarked' column, and prints the number of survivors and non-survivors by gender. The code is as follows:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8 # 1. Get the number of survivors by gender
9 print(data[data['Survived']==1]['Sex'].value_counts())
10
11 # 2. Get the number of non-survivors by gender
```

Test Runner: The test runner shows the results of a single test case. The test case is named "Test case 1" and has a duration of 63ms. The test passed, as indicated by the green checkmark and the message "1 out of 1 shown test case(s) passed". The test case details are as follows:

Expected output	Actual output
female: ...233	female: ...233
male: ...109	male: ...109
Name: Sex, dtype: int64	Name: Sex, dtype: int64
male: ...468	male: ...468
female: ...81	female: ...81
Name: Sex, dtype: int64	Name: Sex, dtype: int64

The bottom of the interface includes a terminal, test cases list, and navigation buttons for previous, reset, submit, and next steps.